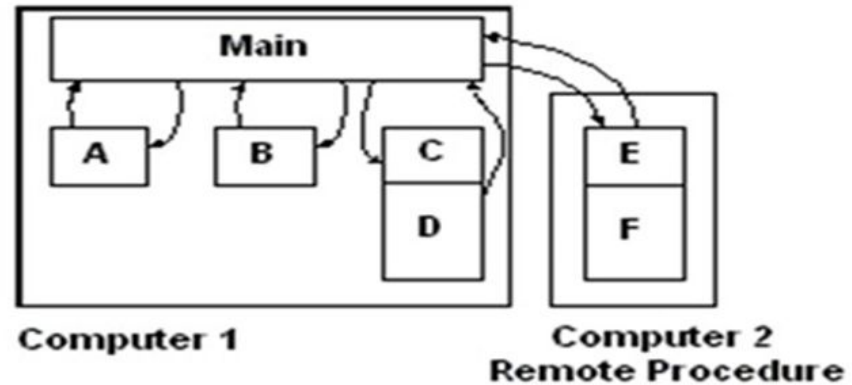
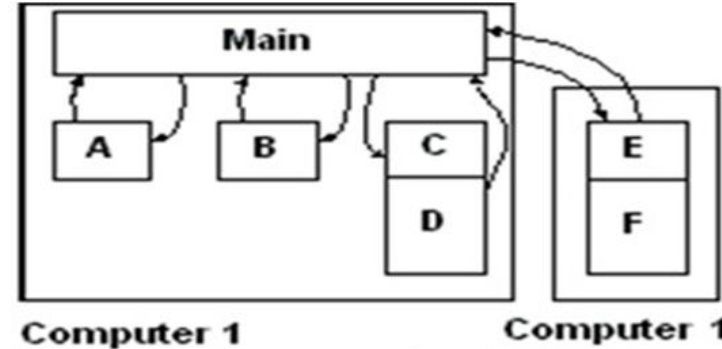
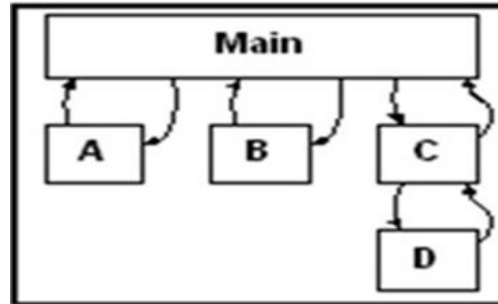


Module 1 - Remote Procedure Call

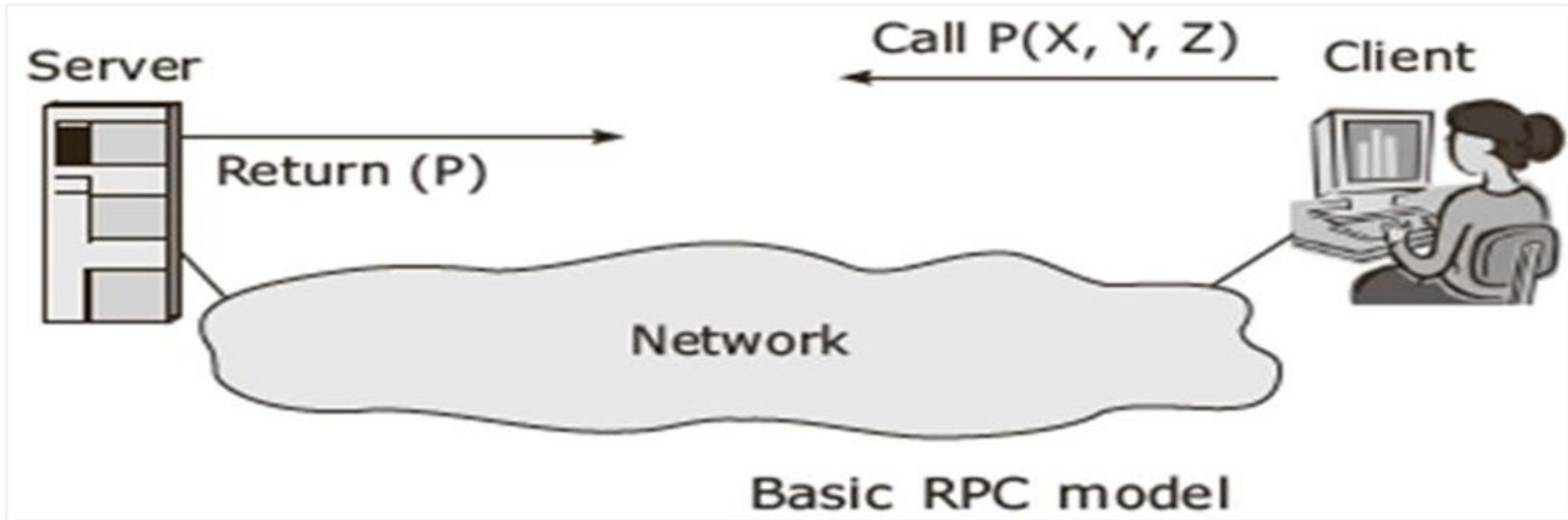
Module 1 - Introduction to Distributed Computing Concepts

No.	Topics
1	Introduction to Distributed Computing Concepts: Basic concepts of distributed systems, distributed computing models, issue in designing distributed systems
2	Inter Process Communication: Fundamental concepts related to inter process communication including message passing mechanism, Concepts of group communication
3	Remote Communication: Remote Procedure Call (RPC), Remote Method Invocation (RMI)

The Programming Model



Remote Subroutine

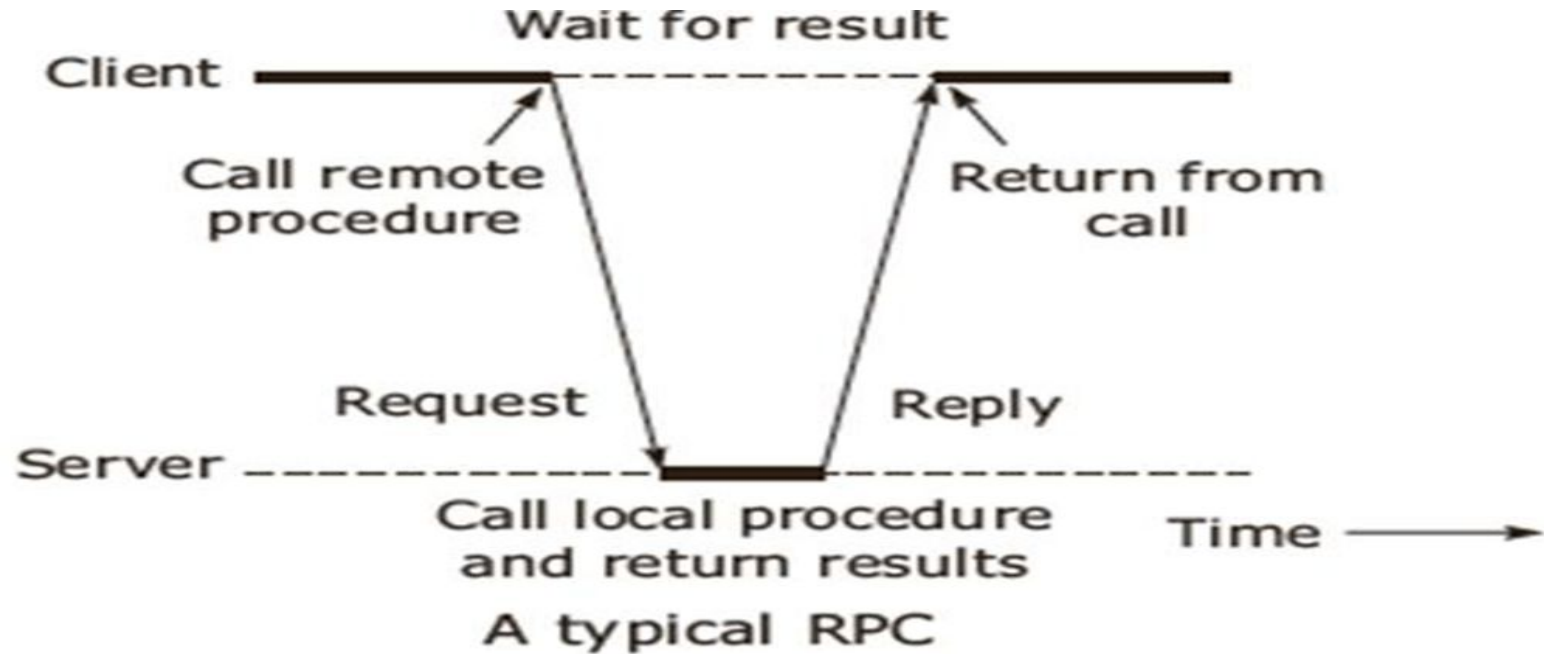


RPC Model

- **Extension of the procedure call mechanism**; Called procedure(remote procedure) can be on same or remote computer.
- The **remote procedure has no access to data and variables** of the caller's environment, as the caller and the callee processes have disjoint address spaces.
- Message-passing scheme is used for information exchange between the caller and the callee processes.



A Typical RPC



RPC

- Remote Procedure Call (RPC) is an efficient, and secure interprocess communication technique for creating distributed client/server programs. It makes client/server interaction easier and safer by **handling common tasks, such as security, synchronization, data flow**.
- It enables data exchange and invocation of functionality residing in a different process. The different process can be on the same machine, on the local area network, or across the Internet.
- RPC(Remote Procedure Call) is a way to
 - **hide communication details behind a procedural call** so user does not have to understand the underlying network technologies
 - **bridge heterogeneous environments**



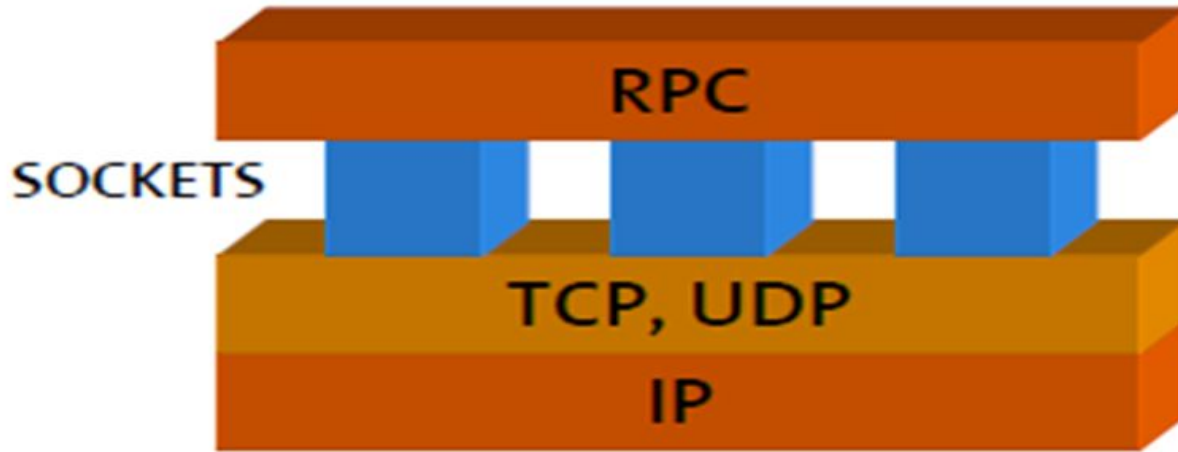
RPC

- Its popularity is mainly because of the following features:
 - **Simple call syntax**
 - **Familiar semantics** (similar to local procedure calls)
 - **Ease of use**: clean and simple semantics of a procedure call makes it simpler to build distributed applications right
 - Its **generality** between single machine and multi machine applications
 - **Efficiency**. Procedure calls are simple enough for communication to be rapid
 - Can be used as an IPC mechanism to communicate between processes on different machines as well as between processes on the same machine

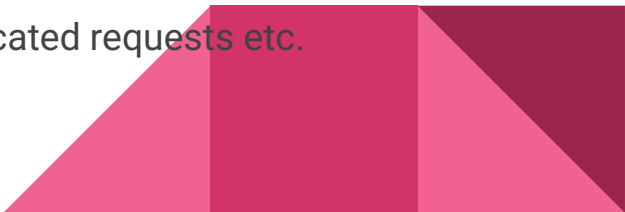


RPC

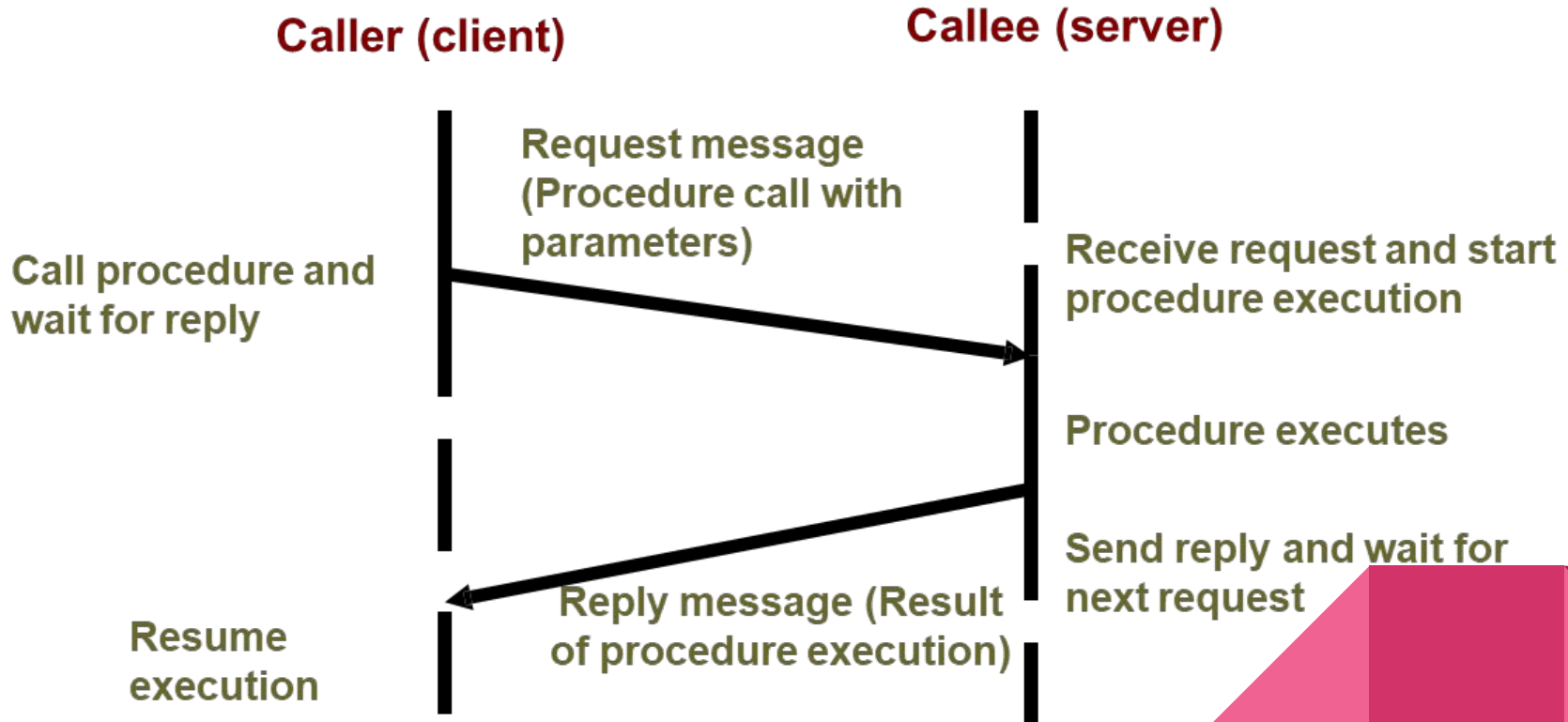
- RPC presumes the existence of a low-level transport protocol, such as TCP (Transmission Control Protocol) or UDP (User Datagram Protocol), for carrying the message data between communicating programs.



Issues Handled by RPC

- How to invoke service in a more or less transparent manner.
 - How to exchange **data** between machines that might use different **representations** for different data types.
 - data type formats (e.g., byte orders in different architectures)
 - data structures (need to be flattened and the reconstructed)
 - How to **find the service** one actually wants among a potentially large collection of services and servers.
 - Client should not know where the server resides or even which server provides the service.
 - How to deal with **errors in the service invocation**:
 - server is down, communication link broken, server busy, duplicated requests etc.
- 

RPC Model



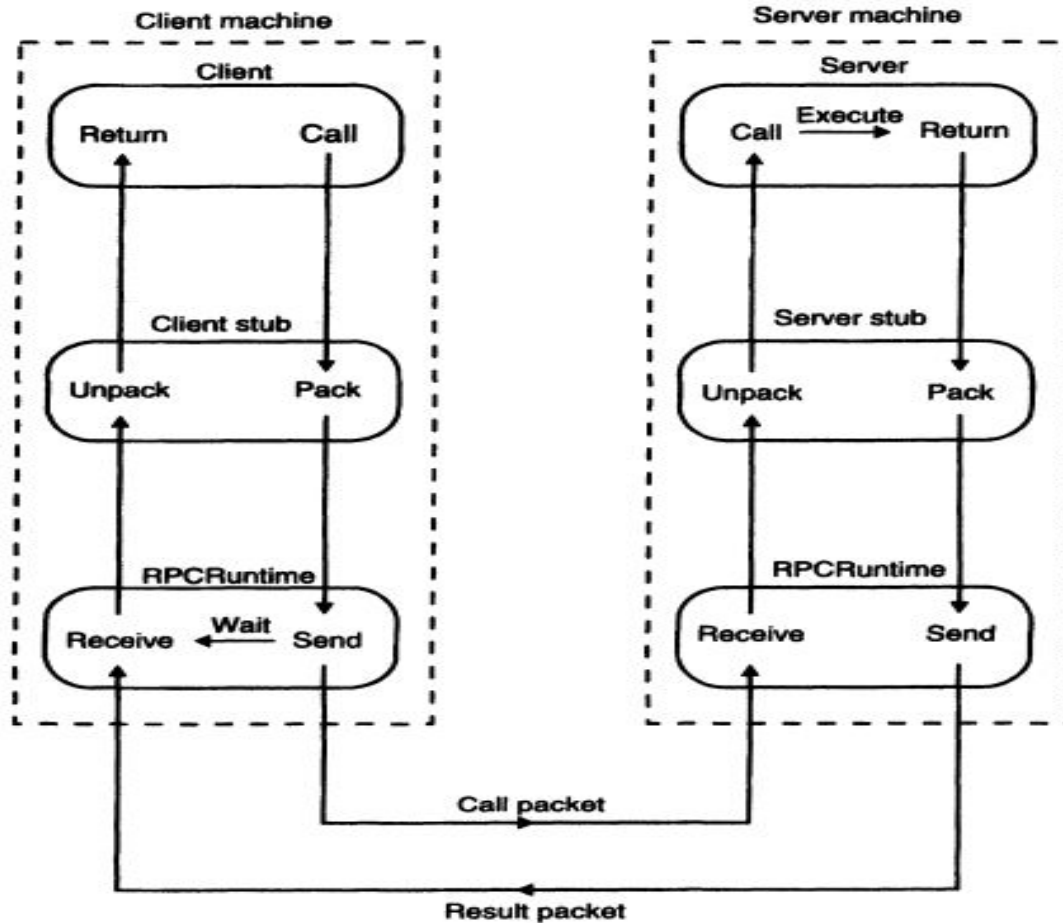
Transparency of RPC

- Local procedures & remote procedures are indistinguishable to programmers.

This requires:

- **Syntactic Transparency** - same syntax as a local procedure call.
 - **Semantic Transparency** - semantics (way of communication) should be same as local procedure call.
 - Semantic Transparency difficult to achieve because:
 - **Address space** of calling process & called process is **disjoint**. Called procedure can not have access to any variable in calling program's environment.
 - Remote procedure calls need the ability to take care of **possible processor crashes & communication problems** of network.
 - Remote procedure calls take **more time** due to involvement of communication network.
- 

How RPC Works (RPC Model)



How RPC Works

- Elements of RPC
 - The client
 - The client stub
 - The RPCRuntime
 - The server stub
 - The server
- To conceal the underlying RPC system from both client & server processes, a separate stub procedure is associated with both processes. It **provides local procedure call abstraction**.
- RPC communication package, **RPC Runtime** hides the existence & functional details of underlying network.



Elements of RPC

- Client

- A process that **requests a service provided by another program**. Client process uses the requested service without having to deal with many working details about the other program or service.

- Client Stub

- Module within a client application containing all of the functions necessary for the client to make remote procedure calls using the model of a traditional function call in a standalone application. Performs following functions :–
 - On a request from client, **packs a specification of target procedure & arguments into a message** & asks local RPC Runtime to send it to server stub.
 - On receipt of procedure execution, it **unpacks result & passes it to client**.
- 

Elements of RPC

- **RPC Runtime**

- Handles transmission across the network between client and server machine.
- Responsible for retransmissions, acknowledgment, packet routing & encryption.

- **Server Stub**

- Module within a server application or service that contains all of the functions necessary for the server to handle remote requests using local procedure calls.
- On receipt of call request, it unpacks it and makes a normal call to invoke the procedure in the server.
- On receipt of the result of procedure execution, it packs the result into a message and ask the RPC Runtime to send it to the client stub.

- **Server**

- A process, such as a program or task, that responds to requests from a client.

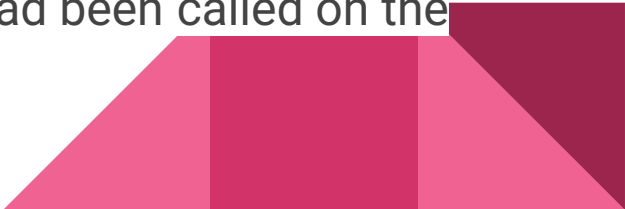
RPC Procedure (1)

1. Client makes a procedure call.
 2. Client stub retrieves the required parameters from the client address space.
 3. Translates the parameters as needed into a standard format (NDR - Network Data Representation) for transmission over the network.
 4. Calls functions in the RPC client run-time library to send the request and its parameters to the server.
 5. The server RPC run-time library functions accept the request and call the server stub procedure.
- 

RPC Procedure (2)

6. The server stub retrieves the parameters from the network buffer and converts them from the network transmission format to the format the server needs.
 7. The server stub calls the actual procedure on the server.
 8. The remote procedure then runs, possibly generating output parameters and a return value. When the remote procedure is complete, a similar sequence of steps returns the data to the client.
 9. Remote procedure returns its data to server stub.
 10. Server stub converts output parameters to format required for transmission over the network and returns them to the RPC run-time library functions.
- 

RPC Procedure (3)

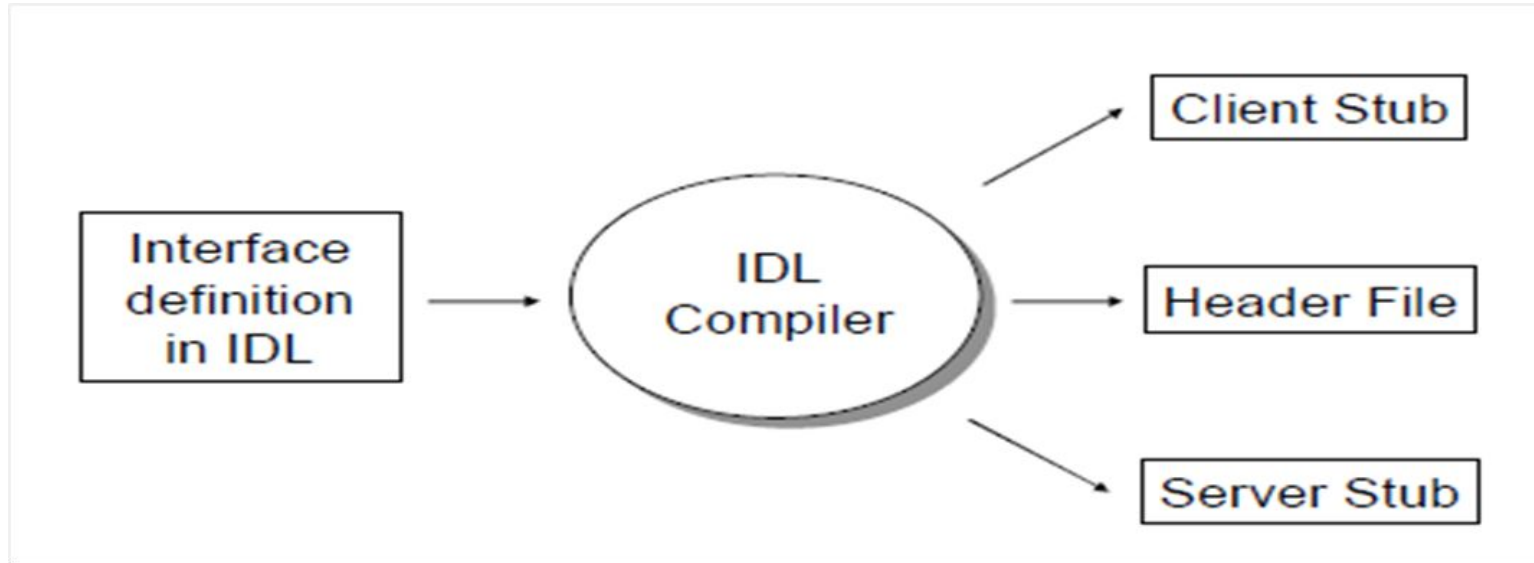
11. The server RPC run-time library functions transmit the data on the network to the client computer.
 12. Client RPC run-time library receives remote-procedure return values & returns them to client stub.
 13. Client stub converts data from standard format to the format used by the client computer. The stub writes data into the client memory and returns the result to the calling program on the client.
 14. The calling procedure continues as if the procedure had been called on the same computer.
- 

Stub Generation

- Manually
 - A set of translation functions are provided from which user can construct his own stub.
- Automatically
 - Uses **Interface Definition Language** to define interface between client & server. It is mainly a list of procedure names supported by the interface, together with their types of their arguments & results.



Where Stubs come from



Interface Definition Language

- IDL **is a purely declarative language**. Defines only types and procedure headers.
- IDLs describe an interface in a **language-neutral way**, enabling **communication between software components that do not share a language**
 - for example, between components written in C++ and components written in Java.



Marshaling Arguments and Results

- Marshalling is **packing of procedure parameters into a message packet(Encoding/ Decoding)**. It involves:
 - Taking arguments or result.
 - Conversion of program objects (language level data structures) into stream - encoding
 - Reconstruction of program objects from stream – decoding
- Marshaling procedures are of two types
 - Provided as **part of RPC software**
 - Used for scalar & compound data types
 - **Defined by users** of RPC system
 - Used for user-defined data types & pointer type



Stateful vs Stateless Server

- **Stateful Server**

- Relieve client from keeping track of state information.
- Fewer disk accesses.
- If a file is opened for sequential access, can read ahead the next blocks.
- If server crashes, client can get inconsistent results.
- If client crashes, its information held by server may no longer be valid.

- **Stateless Server**

- Better equipped to handle failure



Server Creation Semantics

- RPC servers can be classified on the basis of time duration for which they survive.
 - Instance-per-call servers
 - Instance-per-transaction /session servers
 - Persistent servers

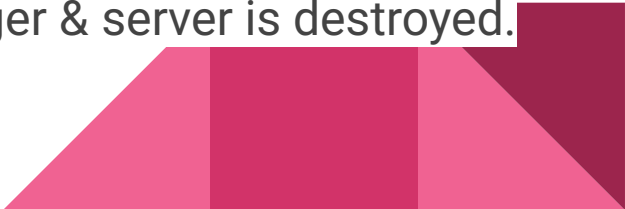


Instance-per-call servers

- Exist only for duration of single call.
- Created by RPC Runtime on the server machine only when call message arrives.
- Deleted after call execution
- Server is exclusively used by a **single client**.
- **Disadvantages**
 - They are **stateless**. Thus intercall state information has to be maintained by either client process or the supporting OS.
 - **Overhead in server creation/ deletion** if same type of service invoked several times successively.



Instance-per-Session Servers

- **Exist for entire session** for which a client & a server interact – can **maintain intercall state information**.
 - **Server managers** for each type of service are registered with the **binding agent**.
 - Client specifies the type of service required to binding agent, which returns address of appropriate server manager.
 - Client contacts the concerned server manager, which **creates a new server** and passes back the address to the client.
 - When session gets over, client informs server manager & server is destroyed.
 - Server is exclusively used by a **single client**.
- 

Persistent Server

- Remain in existence indefinitely.
- Can be shared by many clients.
- Created and installed before the clients.
- Each server registers its service with the binding agent.
- When client contacts binding agent for a particular type of service, it returns address of appropriate server to the client.
- Server interleaves requests from a number of clients, so has to concurrently manage several sets of state information.



Parameter-Passing Semantics

- **Call by value**

- All parameters are copied into a message that is transmitted from the client to the server.
- Not suitable for passing parameters involving voluminous data.
- Usually used as client & server exist in different address space.

- **Call by reference**

- Passing pointers or passing parameters by reference is meaningless in distributed systems.
- Used in distributed shared memory systems or in closed systems, where a single address space is shared by all processes in the system.



Call Semantics

- Call semantics of an RPC system determines how remote procedure may be executed in case of failure.
- Possibly or Maybe call Semantics
- Last-One call Semantics
- Last of Many call Semantics
- At least once call Semantics
- Exactly Once call Semantic



Call Semantic

- Possibly or Maybe call Semantics
 - Weakest semantics - no fault tolerance measures
 - Not really appropriate to RPC
 - In order to prevent the caller from waiting indefinitely for a response from the callee, a time out mechanism is used
 - Caller waits for predetermined timeout period & then continues with its execution.
 - No guarantee of receipt of message.



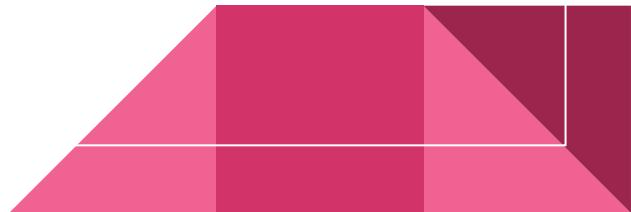
Call Semantics

- Last-One call Semantics

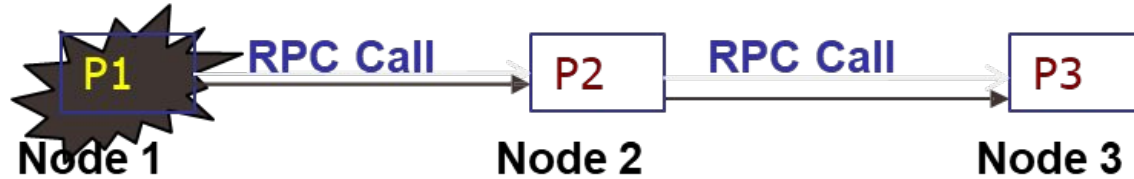
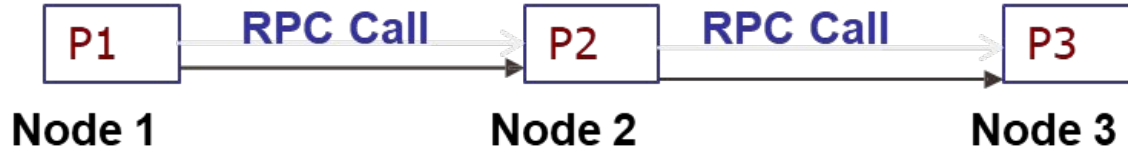
- Client retransmits requests based on timeouts until a response is received by it.
- Result of last executed call is used by the caller.
- Easily achieved only when two nodes are involved in the RPC, but it is tricky to implement it for nested RPCs and cases by orphan calls. An Orphan Call (Orphan RPC) occurs when the client process that made the RPC request terminates, crashes, or loses interest in the request, but the server continues executing the requested procedure.

- Last of Many call Semantics

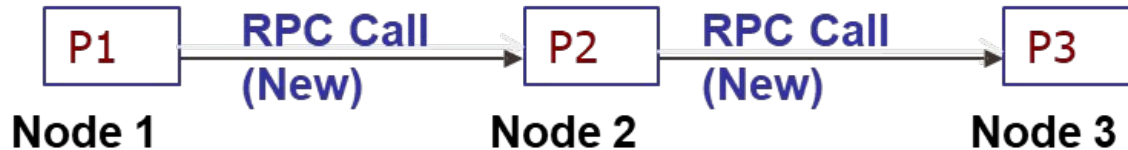
- Similar to last – one semantics except orphan calls neglected.
- When a call is repeated, it is assigned a new call identifier
- Caller accepts a response only if its call identifier matches with identifier of most recently repeated call.



Orphan Calls & Nested RPC



P1 crashes & recovers
& resends request



N3 may return the results
of the two executions of
P3 in any order, possibly
violating last-one
semantics.

These **orphan calls** must be terminated before restarting the crashed processes.

Call Semantics

- **At least once call Semantics**

- Guarantees one or more executions by using timeout based retransmissions.
- **Caller accepts first response message.**

- **Exactly Once call Semantic**

- Eliminates the possibility of a procedure being executed more than once no matter how many times a call is retransmitted.



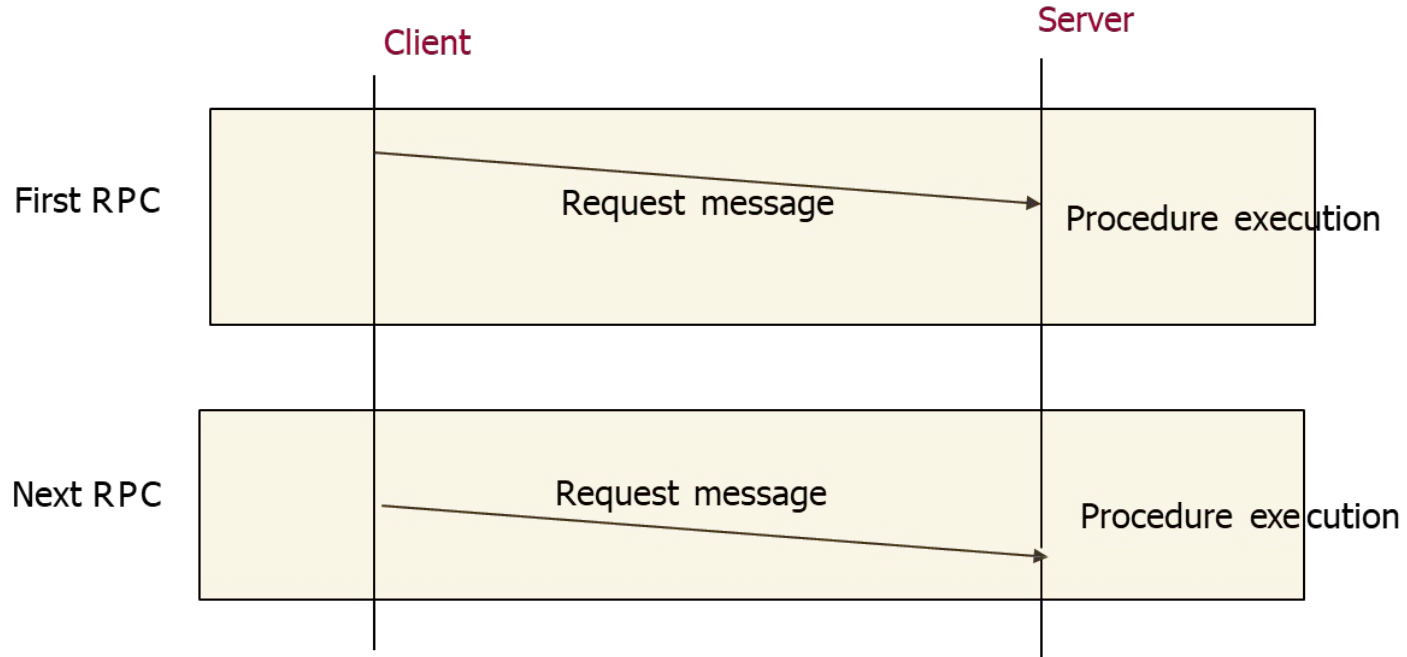
Communication protocol for RPC

- Request (R) Protocol
 - No acknowledgement/ reply required
 - Client not blocked on sending request
 - 1 message per call
 - May-be call semantics
 - No retransmission of request
 - Asynchronous – improves server (no reply generation) / client (not blocked) performance
 - Ex: Periodic update services



Communication Protocol for RPC

Request (R) Protocol

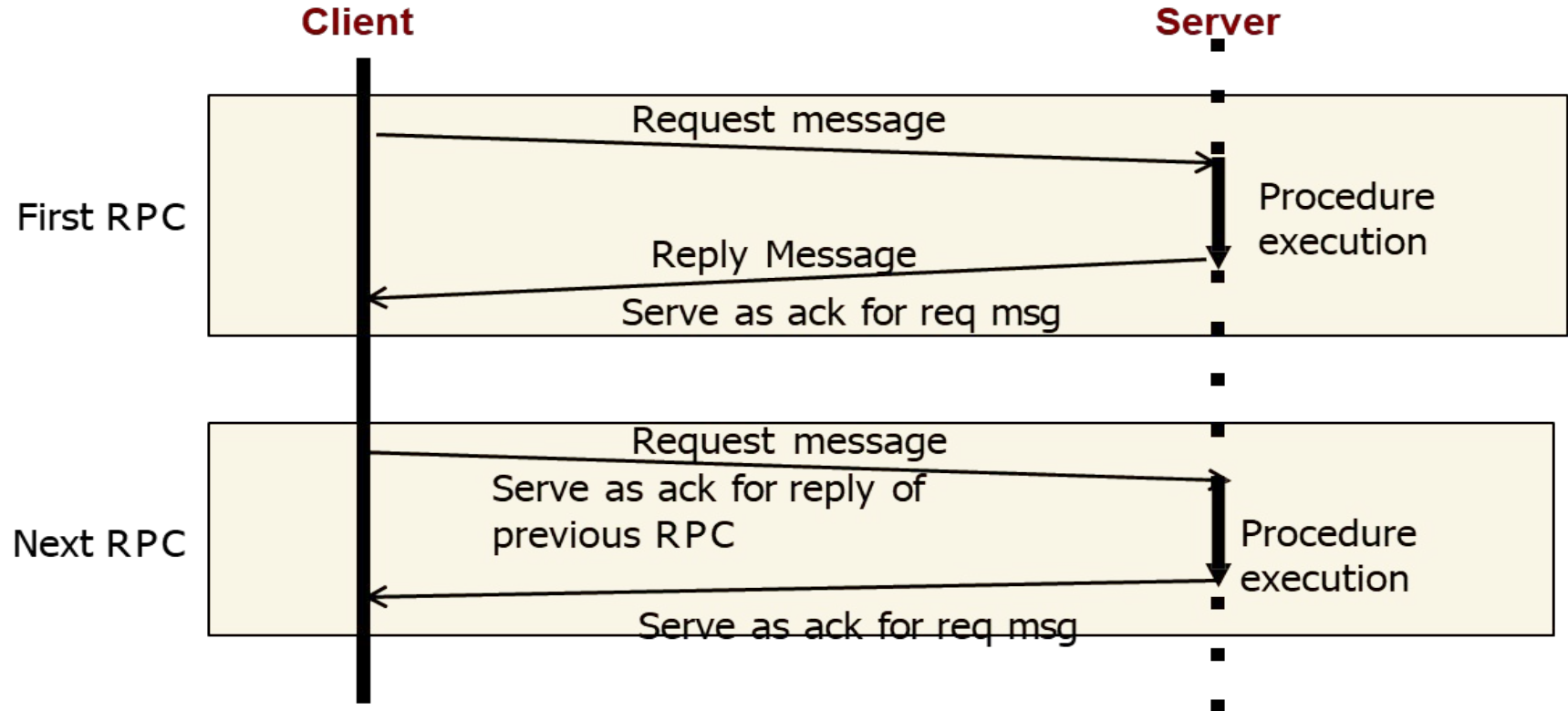


Request/Reply (RR) Protocol

- **Implicit acknowledgement**
- Transmit 2 packets per call.
- Used for **simple RPC's** (RPC in which all arguments & results fit in a single packet buffer & duration of a call & interval between calls are short).
- Client **retransmits request** if it does not receive response message before a predetermined timeout period elapses. Can lead to duplicate executions.
- **At least once or exactly once call** (if use reply cache) semantics.



Request/Reply (RR) Protocol

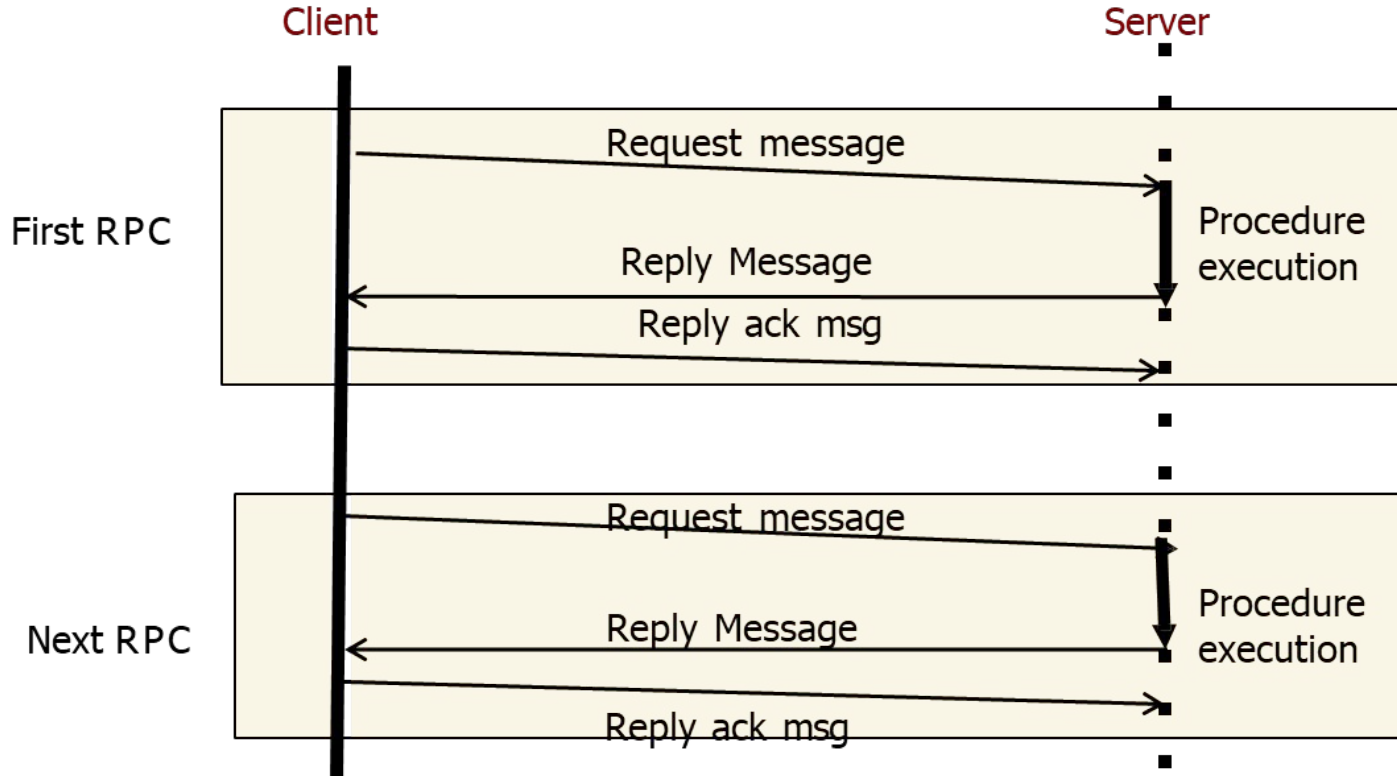


Request/Reply/Acknowledge - Reply (RRA) Protocol

- Three messages per call
- Difficult to maintain reply cache if multiple clients, so this helps implement **exactly once call semantics**
- Server **deletes reply of a request from reply cache only after it gets its acknowledgement**
- All three messages have **same message identifier**
- Loss of acknowledgement message harmless as it is cumulative



Request/Reply/Acknowledge - Reply (RRA) Protocol



Client-Server Binding

- The process by which a **client becomes associated with a server** so that calls can take place is known as binding.
- Binding means locating the server that provides the requested service.
- Various issues:
 - How does client specify server with which it wants to bind?
 - How locate specified server?
 - When to bind client with server?
 - Can binding change during execution?
 - Can client bind with multiple servers that provide same service?



Server Naming

- How does client specify server to which it wants to bind with?
- Specification by a client of a server – uses interface name
- Interface name – type & instance (optional).

// file_server, instance of file server

- Interface name is **unique identifier** of server.
- Type of interface also has a version number to distinguish between old & new versions of interface.



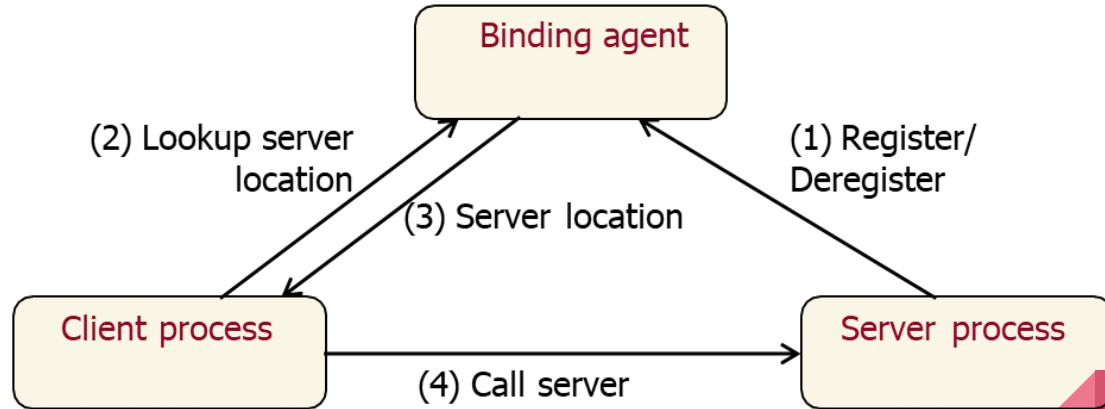
Server Locating

- How locate specified server?
- **Broadcasting** –
 - A message to locate the desired server is broadcasted to all the nodes from the client node.
 - Suitable for small networks.
- **Binding agent** –
 - A Binding Agent is a service that maintains information about the available servers and helps clients locate the appropriate server before making an RPC call.
 - **Name server** which is used to bind a client to a server by providing the client with location information of the desired server.
 - Binding table is maintained to do **the mapping of interface name and its Locations**.
 - All servers register themselves with binding agent as part of their initialization process.



Binding Agent

- Binding agent's location is known to all nodes. Fixed address or broadcasted to all.



Binding Agent

- **Advantages**

- Can support **multiple servers** having same interface type. Fault tolerance.
- Load balancing
- Servers can specify list of **clients authorized** to use its services.

- **Disadvantages**

- **Overhead** in binding clients to servers
- Binding agent **centralized** entity



Binding Agent

- When to bind client with server?
 - Compile Time - Before running the program
 - Link Time - Before starting the program
 - Call Time - Exactly when the client makes the RPC call



Binding Time

- **Compile Time**

- Server's network address can be programmed into the client code i.e. **specify server name in program itself.**
- Inflexible – If server moves or server is replicated, all client programs using that server have to be recompiled.

- **Link Time**

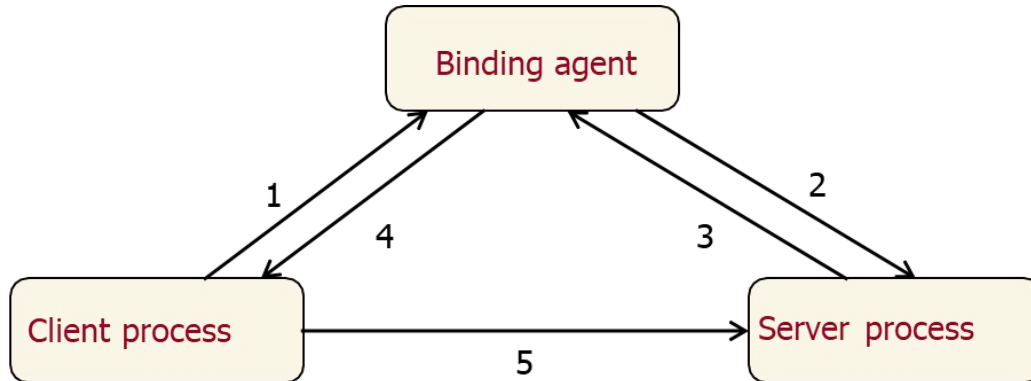
- A client makes an import request to Binding Agent before making a call.
- The Binding Agent binds Client and Server by returning server's handle to the client.

- **Call Time**

- A client is bound to a server, when it calls the server for the first time during its execution.
 - Uses **indirect call method.**
- 

Binding by Indirect Call

1. Client process passes server's interface name & arguments to binding agent
2. Binding agent sends an RPC call message to sever including client's parameters
3. Server returns result to binding agent
4. Binding agent returns result to client with server's handle
5. Client calls server directly subsequently.



Client Server Binding

- **Changing Bindings**

- **Client/ server can change binding** due to some change in system state.
- When binding is altered by concerned server, it is ensured that any state data held by server is no longer needed or can be duplicated in the replacement server.

- **Multiple Simultaneous Binding**

- Client can be bound simultaneously to all or multiple servers of the same type. Ex. Client updates file replicated at many nodes.
- Client uses multicast communication.



Complicated RPC

- RPCs involving long-duration calls or large gaps between calls.
- What if server crashes in between?
 - Periodic probing (Keep checking whether the server is alive) of server by the client helps detect server crash or communication link failure. Message identifier included in probe packet.
 - Periodic generation of an acknowledgment by the server is done if duration of call is long.



Complicated RPC

- RPCs involving arguments/results that are too long to fit in a single-datagram packet.
 - Several physical RPC's for one logical RPC. Overhead involved with each RPC.
 - Use multidatagram messages. Single acknowledgement message for all packets.



Some Special Types of RPC

- Callback RPC
- Broadcast RPC
- Batch mode RPC
- Lightweight RPC

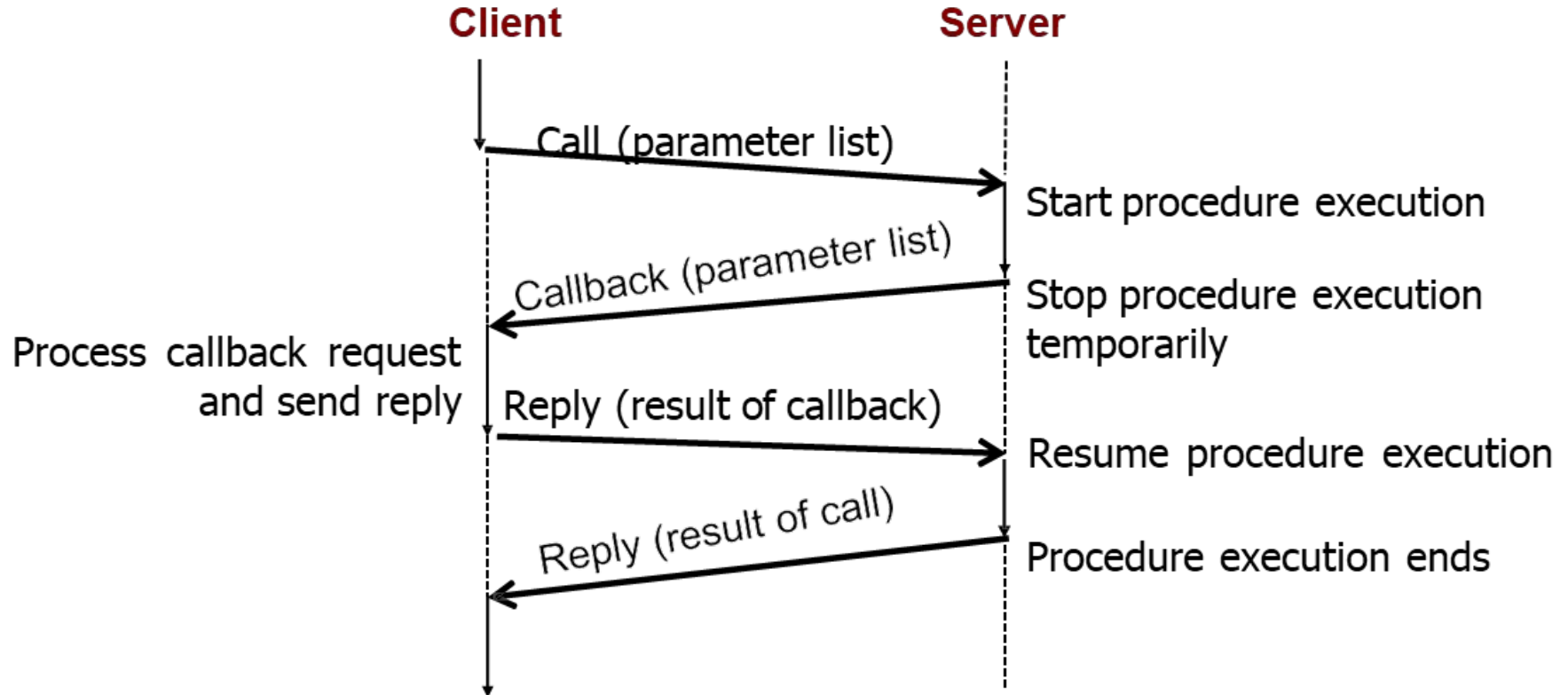


Callback RPC

- In the usual RPC protocol, the caller and callee processes have a client-server relationship
- Facilitates **peer-to-peer paradigm** by allowing process to be both client and a server
- **Server may make several callbacks to client** before returning result of the initial call to the client process.
- For example a remotely processed interactive application that need user input from time to time for further processing, require this facility



Callback RPC



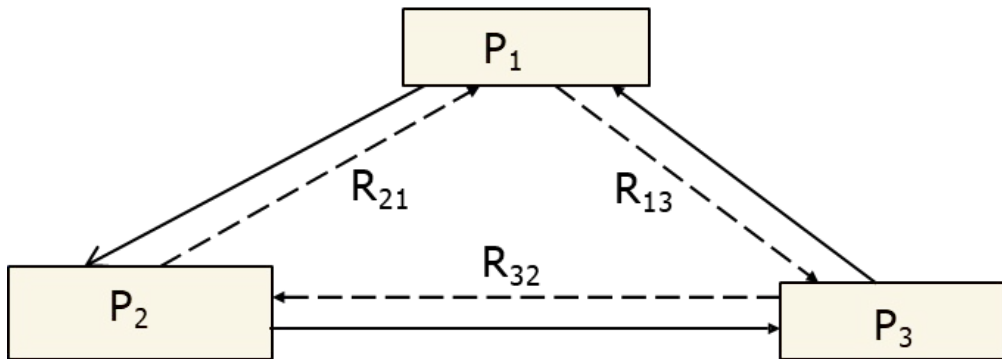
Essentials of Callback RPC

- Providing the **server with the client's handle**
 - The client process uses a **temporary program number** for the callback service and **registers it with Binding Agent**, which is then sent as part of RPC request to the server.
 - To make a callback RPC , the server initiates a normal RPC request to the client using the given program number.
- Making the **client process wait**
 - Client makes a call to **svc-routine**, which waits for server request & then dispatches request to appropriate procedure.



Essentials of Callback RPC

- Handling **call back deadlocks**.
 - A callback deadlock can occur due to the interdependencies of processes.



P_1 is waiting for R_{21} (reply from P_2 to P_1)
 P_2 is waiting for R_{32} (reply from P_3 to P_2)
 P_3 is waiting for R_{13} (reply from P_1 to P_3)

Broadcast RPC

- A client's request is broadcasted on the network and is processed by all the servers that have the concerned procedure for processing that request. The client waits for & receives many replies depending on **degree of reliability** desired.
 - Client uses **special broadcast primitive**. Request sent to **binding agent**, which forwards request to all servers registered with it.
 - To **declare broadcast ports**. Client of broadcast RPC obtains a binding for a broadcast port & then broadcasts RPC message by sending RPC message to this port.



Batch-Mode RPC

- Used to **queue separate requests in a transmission buffer** on the client side and then send them over the network in one batch to the server.
- **Reduces overhead** of sending single RPC.
- **Applications requiring higher call rates** (50 -100 calls per sec) become feasible with the use of batch-mode RPC.
- Used when client has many RPC requests to send to a server & client does not need any reply for a sequence of requests.



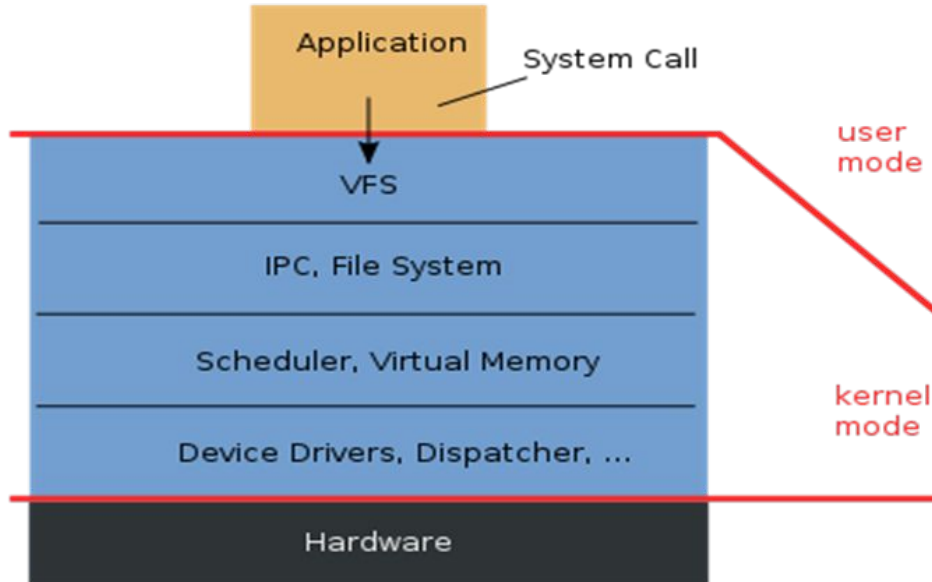
Batch-Mode RPC

- Queue of requests flushed when :-
 - A **predetermined interval** elapses.
 - A **predetermined number of request** have been queued.
 - Amount of batched data **exceeds the buffer size**.
 - A call is made to one of the server's procedures for which a **result is expected**.

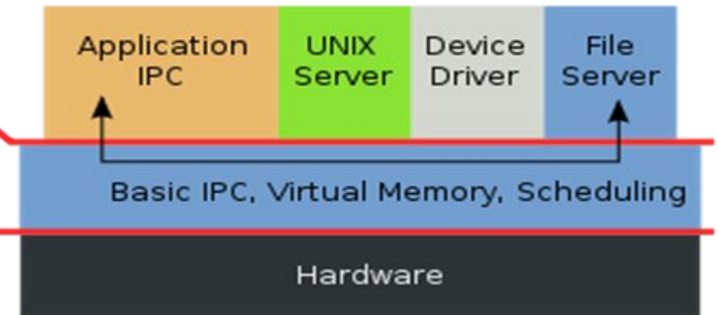


Monolithic vs. Micro Kernel

**Monolithic Kernel
based Operating System**



**Microkernel
based Operating System**



Lightweight RPC

- In microkernel approach, the communication traffic in OS are of two types:
 - **Cross domain** – communication between domains on the same machine
 - **Cross machine** - communication between domains located on separate machines.
- **Lightweight RPC is used for cross- domain communications.**
- In cross domain communication **small-simple parameters are involved**, overhead for the heavyweight machinery (message buffer, marshaling, message transfer, access validation, scheduling, dispatch, etc) is still paid.



RMI (Remote Method Invocation)

Remote Method Invocation

- Extension of procedure call mechanism.
- Allows object residing in one system to gain access/invoke an object running on another system.
- Responsible for remote communication between processes.



Goals to be achieved - RMI

- Minimize complexity.
- Preserve type safety.
- Minimize semantic complexity.

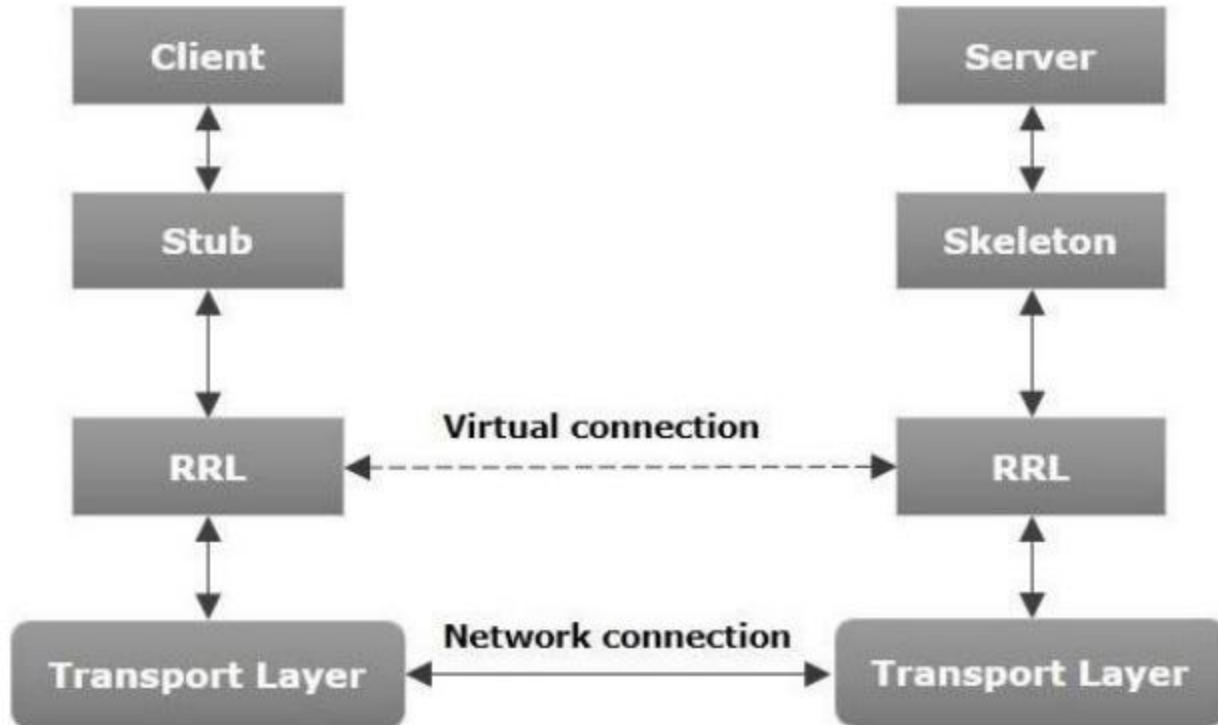


Architecture

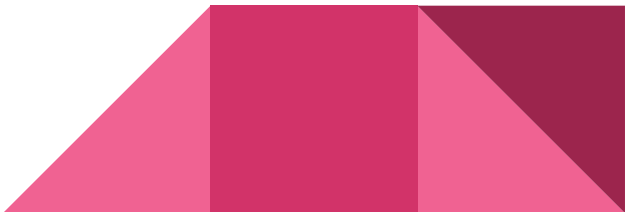
- Server and client are the main feature of an RMI application.
- **Client** - requests remote objects and tries to invoke its methods.
- **Server** - remote object is created and access is given to the client.
- **Elements** of RMI -
 - Transport Layer
 - Stub
 - Skeleton
 - RRL (Remote Reference Layer)



Architecture



Architecture

- **Client**
 - A process that requests a service provided by another program.
 - **Client Stub**
 - Module within a client application containing all of the necessary functions for the client to make remote procedure calls.
 - Transforms client request to message, performs **marshalling**.
 - **Skeleton**
 - Resides on the server side. Client Stub communicates with skeleton to pass request to the remote object.
 - **RRL (Remote Reference Layer)**
 - **Transport Layer**
- 

Architecture

- **RRL (Remote Reference Layer)**
 - This layer manages the references made by client to the remote object.
- **Transport Layer**
 - Handles connections between client and the server.
 - Responsible to manage new and existing connections.



Working of RMI

- When the **remote object gets call** from the client object, it is **received by the stub** and after that it eventually **passes this request to the RRL**.
- After getting request from the remote object to **the client-side RRL**, it invokes a method. RRL gets request from the server side.
- After passing the request on the server side it passes the request to the Skeleton which finally invokes the required object on the server.
- The result is passed all the way back to the client



Marshalling and Unmarshalling

- Marshalling is **packing of procedure parameters into a message packet(Encoding/ Decoding)**. It involves:
 - Taking arguments or result.
 - Conversion of program objects (language level data structures) into stream - encoding.
 - Reconstruction of program objects from stream – decoding.

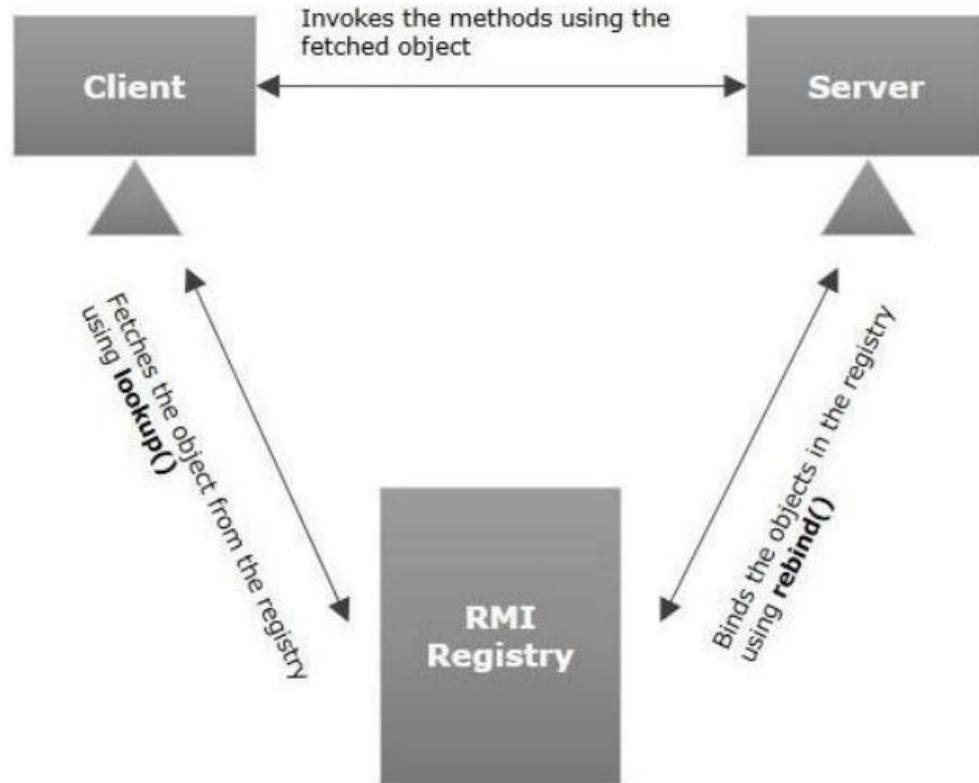


Binding in RMI

- Binding in RMI is done through **RMI Registry**.
- RMI registry is a namespace on which all server objects are placed.
- Every server object has to **register itself on creation/initialization**.
- These are registered using a unique name known as **bind name**.
- To invoke a remote object, the client needs a reference of that object.
- The client **fetches** the object from the **registry using its bind name**.



RMI Registry



Case Study - Self Study

- Suggested reading –
 - RMI in JAVA
 - <https://www.javatpoint.com/RMI>
 - https://www.tutorialspoint.com/java_rmi/java_rmi_introduction.htm
 - Sun RPC
 - <http://www.cdk5.net/rmi/Ed2/SunRPC.pdf>



Questions

- Short notes – Marshalling, RPC, Lightweight RPCs
 - Server creation semantics involved in RPC.
 - Role of binding agent in client server communication.
 - Difference between Stateful/ Stateless servers. Where stateful preferred?
 - What is an orphan call? How implemented in i) last one call semantics ii) last of many calls iii) at least once
 - Difference between local & remote procedure call. RPC model with diagram.
 - What is stub? How are stubs generated? Explain how they make RPC transparent?
 - Why do most RPC systems support call by value semantics for parameter passing?
 - Callback RPC.
 - Difference – Monolithic & Micro kernel
- 